Arlette Chêne
M1 Data Engineer

PROJET IA – Revue de code (code 11)

Objectif :

L'idée est de faire une IA qui fait de la revue de code sur les points suivants :
- Explication des erreurs rencontrées
- Vérification de la cohérence de l'écriture
- Proposition d'optimisation de code

L'IA utilisera :
- Des fichiers pdf d'explication algorithmique
- Utilisera du search pour expliquer les problèmes

Language reviewed :
- python : Comme je l'utilise beaucoup, je vais d'abord utiliser l'IA sur python, un langage ou je peux contrôler les outputs de l'IA.

Objectif bonus :
- Language C : le Saint langage C.
- Implémentation VsCode

Language de programmation :
- python : Python est très utilisé avec l'IA et avec de nombreuses librairies. L'avantage derrière, surtout dans un lapse de temps relativement court, c'est de pouvoir utiliser justement ces librairies, ce qui évite de devoir recréer quelque chose qui existe déjà et qui est partagé. De plus, je maîtrise plutôt bien python, j'aime beaucoup la simplicité de ce langage, et pour créer quelque chose que je ne connais pas ou mal, cela évite aussi de perdre du temps avec la syntaxe et les spécifications d'un langage.

Environnement :
- Code – OSS : IDE qui est en fait VSCode mais sans la télémétrie Microsoft. Ayant débuté sur notepad++ que j'utilise encore parfois, je trouve que l'intérêt premier de Code – OSS c'est toute la communauté qu'il y a derrière et qui propose des extensions utilisables en tout genre, pour tout les langages, mais aussi la possibilité de modifier et d'exporter facilement les paramètres à l'aide d'un fichier json. Derrière il y a aussi l'habitude de l'environnement de développement.
- Conda : installé à l'aide de miniconda3 avec le gestionnaire de package AUR (Arch Unofficial Redepository) ce gestionnaire d'environnement python est parfaitement adapté à beaucoup de situation, notamment à tout ce qui est relié au travail scientifique (matplotlib, sklearn…), mais je trouve aussi la gestion des paquets relativement simple et efficaces vu que conda installe automatiquement les dépendances (et à la bonne version lié à la version de python).
L'autre avantage est que je trouve très facilement comment implémenter la librairie dont j'ai besoin, car parfois certaine librairie nécessite l'utilisation de conda-forge. Dans un moteur de recherche (j'utilise DuckDuckGo qui normallement ne récupère pas les données utilisateur), il suffit de taper *conda + le nom de la librairie.*
Rappel : pour conda, la plupart des librairies que j'utilise sont installés via conda-forge > ***conda install conda-forge ::*nomDeLaLibrairie****

IA utilisée :
- DeepSeek : IA développé par une équipe chinoise, qui récupèrent sans aucun doute les données utilisateurs, mais qui en contrepartie ne demande aucun paiement de la part des utilisateurs. Je la trouve comparable à chatGPT, avec souvent des résultats légèrement meilleur, mais légèrement plus sensible au prompt.

JOURNAL

02.03.26

Choix du projet : Agent IA pour de la revue de code. Pdf fourni avec l'explication des choix technologiques voulu et l'intention de l'application.

10.03.26

Création avec l'IA d'un plan de développement et de la détermination des outils utilisés

Prompt utilisé :
Je dois créer une IA qui fait de la review de code :
L'idée s'est que l'IA puisse voir ce qui se fait de mieux en terme algorithmique en se basant sur des fichiers et ce que l'utilisateur veut faire (ou sur Internet, mais pour les fichiers se sont des pdf qui viennent d'informaticiens de pointent qui présentent des optimisations de calcul).
Elle devra aussi vérifier la cohérence de l'écriture (pour la fermeture ouverture guillement ou encore les snakeCase ou camel_case).
Elle expliquera aussi les erreurs que peuvent rencontrer le code.
Je ne sais pas comment débuter le projet, je n'ai aucune idée de ou chercher, et j'aimerais que tu m'aides pour ce projet.
Le code du projet sera en python. On se concentrera d'abord sur de la review uniquement sur du code python. Tu devras me faire un PRD et des spec avec un user storie.
Tu débuteras d'abord sur la première étape expliqué pas à pas, et quand je te le dirais on passera à l'étape suivante/

I. PRD

1. Objectif général du produit :

Créer un outil automatisé de revue de code Python qui :

- Analyse la qualité et la cohérence du code (style, conventions, erreurs potentielles).
- Propose des améliorations algorithmiques basées sur des connaissances extraites de documents PDF de référence (articles, cours d'experts).

- Fournit des explications claires sur les problèmes détectés.

Public cible : développeurs, équipes souhaitant automatiser une partie de leurs revues de code, étudiants apprenant les bonnes pratiques.

2. Fonctionnalités principales :

A. Analyse syntaxique et stylistique

- Vérification des conventions de nommage : détection des `snake_case` (recommandé en Python) vs `camelCase` (à signaler).
- Contrôle de la cohérence des guillemets (simples/doubles) dans tout le fichier.
- Détection d'erreurs basiques : variables non utilisées, importations inutiles, etc.
- Génération d'un rapport lisible avec les lignes concernées et une explication.

B. Analyse algorithmique avancée

- Chargement et interprétation de fichiers PDF contenant des descriptions d'algorithmes optimisés (fournis par l'utilisateur ou intégrés dans une base de connaissances).
- Comparaison du code soumis avec des patrons algorithmiques reconnus (ex : remplacer une boucle naïve par une approche plus efficace).
- Proposition de remplacement avec justification (ex : "Cette fonction de tri peut être remplacée par `sorted()` qui est implémenté en C et plus rapide.").

C. Explication des erreurs et suggestions

- Pour chaque problème détecté, fournir une explication pédagogique : pourquoi c'est un problème, quel impact cela peut avoir (performance, lisibilité, fiabilité).
- Suggestion de correction immédiate (quand c'est possible).

D. Interface et utilisation

- Mode ligne de commande (CLI) en premier lieu : `python review.py mon_fichier.py` (ou avec un dossier).
- Possibilité d'intégration future dans un IDE ou un service web.

3. Contraintes techniques :

Langage de développement : Python 3.9+

- Modules externes envisagés :
- Analyse de code : `ast`, `pylint`, `flake8`, `black` (pour le formatage)
- Extraction PDF : `PyPDF2`, `pdfplumber`, ou `camelot` selon le contenu
- NLP pour la compréhension des PDF : `spaCy`, `transformers` (si nécessaire)

- Interface CLI : `argparse` ou `click`
- Tests : `pytest`
- Gestion de versions : Git (dépôt GitHub/GitLab)

4. User Stories :

Les user stories décrivent ce que l'utilisateur veut accomplir avec le produit. Elles seront notre fil rouge pour prioriser les fonctionnalités.

1. En tant que développeur, je veux soumettre un fichier Python à l'outil afin de recevoir un rapport sur les incohérences de style et les conventions de nommage.
2. En tant que développeur, je veux que l'outil détecte automatiquement les erreurs courantes (variables inutilisées, imports superflus) pour gagner du temps en revue manuelle.
3. En tant que développeur cherchant à optimiser son code, je veux que l'outil compare mon implémentation avec des algorithmes optimisés décrits dans des PDF de référence pour améliorer les performances.
4. En tant que responsable d'équipe, je veux pouvoir fournir à l'outil une collection de PDF d'experts (ex : "Introduction to Algorithms" de Cormen, articles spécialisés) afin que l'IA puisse s'enrichir de ces connaissances et proposer des améliorations pertinentes.
5. En tant que développeur junior, je veux que chaque suggestion soit accompagnée d'une explication claire pour comprendre pourquoi c'est mieux et apprendre les bonnes pratiques.
6. En tant que utilisateur, je veux que l'outil soit utilisable simplement en ligne de commande pour l'intégrer dans mon flux de travail actuel (hooks Git, CI).

5. Prioritisation :

Version minimale (MVP) :

- Analyse de base du code Python (AST) pour détecter les erreurs évidentes.
- Vérification des conventions de nommage et guillemets.
- Rapports simples en console.
- Pas d'analyse algorithmique avancée, juste une base de règles prédéfinies.

Version 2 :

- Intégration de l'extraction de connaissances à partir de PDF (premiers tests sur des documents simples).
- Suggestions algorithmiques simples.

Version 3 :

- Amélioration de l'IA via apprentissage automatique sur les PDF.
- Interface graphique ou plugin IDE.

II. Conception Technique et Architecture

1. Architecture générale

- code_review_ai/
  - core/
    - parser.py # Analyse du code
    - style_checker.py # Vérification stylistiques
    - error_detector.py # Détection d'erreurs
    - algorithm_advisor.py # Suggestions algorithmique
  - knowledge/
    - pdf_extractor.py # Extraction depuis pdf
    - knowlegde_base.py # Base de règles/algorithmes
    - matcher.py # Comparaison code / connaissances
  - reporting/
    - formatter.py # Mise en forme
    - explainer.py # Générateur d'explications
  - cli/
    - main.py # Point d'entrée principales
  - tests/ # Tests unitaires et d'intégrations
  - config/ # Configuration (règles, chemin etc.)
  - agent.md # Description de l'agent
  - README.md # Présentation générale du projet
  - PRD.md # Product Requierment
  - setup.py

2. Choix technologiques

1. Analyse du code python

- ast en module standart pour parcourir l'arbre syntaxique
- pylint / pyflakes : pour réutiliser des détections existantes sans tout réinventer. On peut appeler ces outils en sous-processus et parser leur sortie.
- black : pour vérifier le formatage (optionnel, car on peut définir nos propres règles de style).
- **Approche** : on combine l'AST pour des règles sur mesure et on utilise pylint pour les erreurs classiques.

2. Extraction des connaissances sur des pdf

- PyPDF2 : simple, gratuit, mais limité pour les PDF complexes (tableaux, formules).
- pdfplumber : meilleur pour extraire du texte et des tableaux de manière plus fiable.

- Pour l'IA avancée : si les PDF contiennent beaucoup de formules mathématiques ou de pseudo-code, on pourra envisager camelot (tableaux) ou pymupdf (fitz) pour une extraction fine.
- Traitement NLP : si on veut comprendre le sens des descriptions, spaCy permettra d'extraire des entités (noms d'algorithmes, complexités, etc.). Dans un premier temps, on se contentera de mots-clés.

3. Base de connaissance

Stockage des "recettes" algorithmiques sous forme structurée (json)

4. Comparaison code <> connaissance

Deux approches complémentaires :

1. Statique : détection de motifs syntaxiques dans l'AST (ex : présence de deux boucles for imbriquées avec échange d'éléments = probable tri naïf).
2. Sémantique (V2+) : analyse de la complexité algorithmique estimée (nombre d'opérations, structure des boucles) pour suggérer des alternatives.

5. Génération d'un rapport

Sortie console : format lisible avec couleurs (via colorama ou rich).

- Sortie JSON : pour intégration dans d'autres outils (CI).
- explainer.py : pour transformer une erreur technique en explication pédagogique (ex : "La variable x n'est jamais utilisée → cela alourdit le code sans effet, vous pouvez la supprimer.").

6. Interface CLI

Utilisation de argparse (standard) ou click (plus moderne) pour gérer les arguments :

text

python -m code_review_ai review mon_fichier.py [--pdf dossier_pdfs] [--output json]

- Gestion des chemins : fichier unique, dossier récursif, pattern *.py.

3. Flux de traitement (workflow)

Entrée : l'utilisateur lance la commande avec un fichier/dossier Python.

1. Analyse statique :
    - Parsing du/des fichiers avec ast.
    - Application des règles de style (nommage, guillemets) → style_checker.
    - Détection des erreurs basiques → error_detector (via pylint ou règles maison).
2. Analyse algorithmique (si activée et si des PDF sont fournis) :

- Chargement de la base de connaissances.
- Pour chaque fonction pertinente, tentative de correspondance avec un motif connu.
- Si correspondance → suggestion d'amélioration.

3. Génération du rapport :
   - Agrégation de toutes les observations.
   - Ajout d'explications pédagogiques.
   - Affichage/sauvegarde selon le format demandé.

4. Génération d'un pdf (phase avancée)

Pour la version avec apprentissage à partir des PDF, voici comment procéder :

1. **Extraction** : `pdfplumber` extrait le texte et les tableaux.
2. **Prétraitement** : nettoyage, détection de sections ("Algorithmes de tri", "Complexité").
3. **Reconnaissance de patterns** : utilisation d'expressions régulières et de NLP simple pour identifier les noms d'algorithmes, les extraits de pseudo-code, les complexités temporelles ($O(n^2)$, $O(\log n)$, etc.).
4. **Structuration** : transformation en entrée de la base de connaissances.

5. Tests et validation

Tests unitaires : `pytest` pour chaque module (ex : vérifier que `style_checker` détecte bien `camelCase` dans un code donné).

- Tests d'intégration : lancer l'outil sur des exemples de code (bons et mauvais) et comparer les rapports attendus.
- Données de test : créer un petit corpus de fichiers Python avec des erreurs variées, et quelques PDF factices pour tester l'extraction.

6. Plan de développement

Étape	Description	Durée estimée
1	Mise en place du projet (structure, dépôt Git, environnement virtuel)	1 jour
2	Implémentation de l'analyseur AST basique (parcours de fichier)	2 jours
3	Règles de style (nommage, guillemets) + rapports console simples	2 jours
4	Intégration de pylint pour les erreurs courantes	1 jour
5	Base de connaissances manuelle et suggestions algorithmiques simples	3 jours
6	Extraction PDF (première version basique)	3 jours
7	Amélioration du matching code ↔ connaissances	3 jours
8	Tests, documentation, packaging	2 jours

17.03.26 (matin 9h > 12h)

> Création de l’architecture et des fichiers (vide)

> Installation de dépendances avec un environnement conda.

- Utilisation de « conda env export > environment.yml »
- Fichier requierments.txt très probablement inutile

> Implémentation du code dans parser.py

- Création d’un fichier test_parser.py
- Test unitaire >> Erreur recontrée :

```
=================================================================== ERRORS =================================================================== ___ ____________________________________________________ ERROR collecting tests/test_parser.py ____________________________________________________ ImportError while importing test module '/home/Archene/Dev/TPIA_Gateway/tests/test_parser.py'. Hint: make sure your test modules/packages have valid Python names. Traceback: ../../.conda/envs/multiUse/lib/python3.13/importlib/**init**.py:88: in import_module return _bootstrap._gcd_import(name[level:], package, level) ^^^
```

- Nouvelle erreur :

========== test session starts ========== platform linux -- Python 3.13.12, pytest-9.0.2, pluggy-1.5.0 rootdir: /home/Archene/Dev/TPIA_Gateway collected 2 items

tests/test_style_checker.py F. [100%]

========== FAILURES ========== _____ test_naming_issues _____

sample_file_with_issues = PosixPath('/tmp/pytest-of-Archene/pytest-5/test_naming_issues0/sample_style.py')

```
def test_naming_issues(sample_file_with_issues):
    parser = CodeParser(str(sample_file_with_issues))
    checker = StyleChecker(parser)
    issues = checker.check_all()
    naming_issues = [i for i in issues if i['type'] == 'naming']

  assert len(naming_issues) == 3  # maClasse, attributVariable, MaFonction

    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

E assert 2 == 3 E + where 2 = len([{'line': 2, 'message': "Le nom de classe 'maClasse' devrait être en CamelCase (ex: MaClasse).", 'subtype': 'class_cas...MaFonction' devrait être en snake_case (ex: ma_variable).", 'subtype': 'snake_case', 'suggestion': 'ma_fonction', ...}])

tests/test_style_checker.py:26: AssertionError ========== warnings summary ========== tests/test_style_checker.py: 47 warnings /home/Archene/Dev/TPIA_Gateway/core/style_checker.py:57: DeprecationWarning: ast.Str is deprecated and will be removed in Python 3.14; use ast.Constant instead if isinstance(node, ast.Str): # pour Python < 3.8 ; pour 3.8+ utiliser ast.Constant

tests/test_style_checker.py::test_naming_issues tests/test_style_checker.py::test_naming_issues tests/test_style_checker.py::test_quotes_issues tests/test_style_checker.py::test_quotes_issues /home/Archene/Dev/TPIA_Gateway/core/style_checker.py:58: DeprecationWarning: Attribute s is deprecated and will be removed in Python 3.14; use value instead s = node.s

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html ========== short test summary info ========== FAILED tests/test_style_checker.py::test_naming_issues - assert 2 ==

3 ========== 1 failed, 1 passed, 51 warnings in 0.03s ==========

- Deux modifications apportées : parser.py et style_checker.py

  Note : pour la lisibilité, plus de c/c des prints d'erreurs

> Création du fichier error_detector.py et de son fichier test, et son intégration dans le CLI (modification de main.py)

- Erreur : Il manque la librairie pylint > ajout dans le fichier environment.yml. Ajout de pylint avec conda-forge.
- Erreur : Erreur de nouveau avec pylint. Ajout d'un test qui créé un fichier vide, et modification d'un test.
- Erreur : Erreur de captation du fichier json. Modification du code error_detector.py et de son fichier test " OK !!

> Création du fichier knowledge_base.py, matcher.py, algorithm_advisor.py et de leurs fichier test.

- Changement d'emplacement de knowledge_base.py (initialement dans le dossier core)
- Test : Check Ok du premier coup.

> Modification du fichier main.py dans le CLI : intégration du knowledge_advisor.

> Peaufinage des algorithmes : besoin de régénérer plusieurs fois le prompt qui ne va pas jusqu'au bout.

- Parser >>
  - Ajout d'une gestion d'exception pour SyntaxError dans __init__
  - Etendre get_all_names pour inclure les noms dans les paramètres de fonction, les compréhension, les alias d'import etc.
  - Ajout d'une méthode get_string_token qui retourne les tokens de chaines pour une utilisation centralisée.
  - Ajout de la méthode get_functions_detailed qui retourne des infos supplémentaires (nom, ligne début/fin, docstring etc.)
  - Ajout d'une méthode get_lines pour accéder facilement au contenu ligne par ligne.
  - Documenter chaque méthode avec des docstrings claires.
  - Test des modications Ok.
- Style_checker >>
  - Création d'une classe Issue.py pour uniformiser les rapports (nouveau fichier !!) : test OK !
  - Amélioration de la détection des guillements (prise en compte des f-strings, raw strings et triples quotes.
  - Ajout de règles supplémentaires : vérification des espaces en fin de ligne, longueur de ligne etc.
  - Documentation des méthodes avec des docstrings.

    - Tests pour couvrir les nouveaux cas.
    - Test des modications Ok.
- Test rapide de main.py sur un fichier python :
    - Récupération d’un ancien fichier python où j’ai moi même repérés des erreurs, que je renomme python_test.py
    - Code du fichier trouvable dans le dossier tests.
    - Résultat du output :

(multiUse) [Archene@lazychene TPIA_Gateway]$ python -m cli.main tests/python_test.py :52: SyntaxWarning: invalid escape sequence '\ ' 24 problème(s) détecté(s) :

ligne 19 : Le nom 'outPutStr' devrait être en snake_case (ex: ma_variable). Suggestion : out_put_str

ligne 31 : Le nom 'runGame' devrait être en snake_case (ex: ma_variable). Suggestion : run_game

ligne 90 : Le nom 'runGame' devrait être en snake_case (ex: ma_variable). Suggestion : run_game

ligne 50 : Le nom 'outPutStr' devrait être en snake_case (ex: ma_variable). Suggestion : out_put_str

ligne 87 : Le nom 'outPutStr' devrait être en snake_case (ex: ma_variable). Suggestion : out_put_str

ligne 88 : Le nom 'outPutStr' devrait être en snake_case (ex: ma_variable). Suggestion : out_put_str

ligne 64 : Le nom 'outPutStr' devrait être en snake_case (ex: ma_variable). Suggestion : out_put_str

ligne 76 : Espace(s) en fin de ligne. Suggestion : Supprimez les espaces inutiles en fin de ligne.

ligne 77 : Espace(s) en fin de ligne. Suggestion : Supprimez les espaces inutiles en fin de ligne.

ligne 97 : Espace(s) en fin de ligne. Suggestion : Supprimez les espaces inutiles en fin de ligne.

ligne 76 : Trailing whitespace Suggestion : Supprimez les espaces en fin de ligne.

ligne 77 : Trailing whitespace Suggestion : Supprimez les espaces en fin de ligne.

ligne 97 : Final newline missing

ligne 52 : Anomalous backslash in string: '\ '. String constant might be missing an r prefix.

ligne 9 : Missing function or method docstring

ligne 12 : Consider using enumerate instead of iterating with range and len

ligne 19 : Missing function or method docstring

ligne 19 : Function name "outPutStr" doesn't conform to snake_case naming style

ligne 22 : Consider using enumerate instead of iterating with range and len

ligne 31 : Missing function or method docstring

ligne 31 : Function name "runGame" doesn't conform to snake_case naming style

ligne 34 : Using open without explicitly specifying an encoding

ligne 58 : Consider using enumerate instead of iterating with range and len

ligne 31 : Too many branches (15/12)

- Error_Detector
  - Remplacement des dictionnaires par des objets Issue dans les retours
  - Amélioration de la gestion des erreurs et de la fiabilité de l'appel à pylint.
  - Ajout de motifs algorithmiques (recherche dichotomique, tri rapide fait maison, etc...)
  - Amélioration des suggestions et explications.

Explication des modules créés :

- Que fait parser.py ?
  - Analyse un fichier python pour en extraire les informations utiles pour les vérifications ultérieures. Le module standart ast (Abstract Syntax Tree) sera utilisé.
  - Objectif :
    - Lire un fichier python.
    - Construire un arbre syntaxique.
    - Fournir des méthodes pour parcourir les nœuds (classes, fonctions, variables…)
    - Détecter les conventions de nommage et les guillemets, délégués après à style_checker.py

- Que fait style_checker.py ?
  - Vérifie les conventions de nommage et des variables (fonctions et variables en snake_case)
  - Nom de classe en CamelCase
  - Détecter les noms qui ne sont pas en règles et les signaler.
  - Cohérence des guillemets
  - Génération d'observation : les problèmes rencontrés seront remontés sous forme d'un dictionnaire qui contenant la ligne, le type d'erreur, le message et la suggestion de correction.

- Que fait error_detector ?
  - Détecte les erreurs courantes en utilisant pylint > couvre un large éventail de problèmes sans tout réinventer.
  - Objectif :
    - Appel de pylint en sous-processus sur le fichier analysé.
    - Parse la sortie de pylint en format json pour extraire les messages d'erreurs, d'avertissement etc.
    - Converti les messages dans un format interne d'issues.
    - Intègre ces issues dans le rapport final.

- Que fait knowledge_base ?
  - Contient une base de connaissance. Première base de connaissance ajouté manuellement. Ensuite, enrichissement de cette base via l’extraction de pdf.
  - En motif :
    - Un nom (ex : « tri_bulle »)
    - Une description courte.
    - Une fonction de détection (basée sur l’ast)
    - Une suggestion
    - Une explication pédagogique
- Que fait matcher ?
  - Prend une fonction (nœud AST) et retourne les motifs qui correspondent.
- Que fait algorithm_advisor ?
  - Extrait les fonctions et les compare aux motifs. Il génère ensuite des issues de la même façon que les autres modules.

17.03.26 (Après-midi : 16h->17h)

> Reprise de la modification de error_detector :

- Error_Detector
  - Utilisation de la classe Issue pour uniformiser les sorties
  - Meilleur gestion des codes de retour de pulint et des messages d’erreurs.
  - Ajout de suggestions personnalisées pour davantage de symboles.
  - Documentation des méthodes.
  - Premier test : Erreur car issues ne contient pas de symboles attendues ou bien est vide > recode de la gestion de Issue > test OK !
- Algorithm_Advisor :
  - Utilisation de la classe Issue pour uniformiser les rapports.
  - Détecteurs plus robustes (par exemple, gestion des faux positifs ou améliorations des heuristiques)
  - Ajout de motifs algorithmiques (ex : recherche dichotomique)
  - Amélioration des suggestions et améliorations
  - Test Ok du premier coup.

> Premier peaufinage finit en toute beauté : le test de cli.main.py fonctionne correctement.

> Codage de l’extracteur de pdf. Je m’intérroge sur l’extracteur de pdf que je dois utiliser. L’IA m’a d’abord proposé pdfplumber, mais en cherchant sur le net j’ai trouvé ce site : https://dev.to/onlyoneaman/i-tested-7-python-pdf-extractors-so-you-dont-have-to-2025-edition-akm. En lisant les tests, malgré la lourdeur de unstructured, le fait qu’il utilise de l’analyse sémantique, ce qui pour moi est assez important pour pouvoir extraire les connaissances des fichiers pdf. J’ai donc notifié à l’IA que j’allais utiliser cette librairie.

> Rajout de la librairie unstructured avec conda-forge. Je décide finalement d’attendre la fin du projet avant de faire le conda requierment.

> Problème pour l’ajout de la librairie unstructured. Création d’un nouvel environnement conda en python 3.12.

> Pour le NLP (Natural Language Processing) : installation de spacy.

> Je réutilise tous mes tests unitaires pour voir si quelque chose est oublié avec mon environnement : tout est ok. Note : l’environnement précédemment utilisé est mon environnement « couteau-suisse », qui a tendance à devenir inutilisable lorsque je créé quelque chose de très spécifique.

> Création du fichier pdf_extractor.py, et du fichier nlp_extractor.py.

> Ajout d’un fichier knowledge_builder.py puis d’un fichier de test unitaire pour les 3 fichiers python précedemment créé.

> Modification du CLI

> Tentative de trouver un bon fichier pdf qui explique l’algorithmique, mais manque de temps. Reprise plus tard.

- Que fait pdf_extractor ?
  - Il découpe en bloc sémantique un pdf (par exemple titre, paragraphes etc.)
- Que fait nlp_extractor ?
  - Il utilise spacy pour analyser les blocs de textes créé par pdf_extractor. Il récupérera le nom des algorithmes, leurs complexités et leur mot-clé.
- Que fait knowledge_builder ?
  - Il va récupérer les blocs construits par npl_extractor pour générer des motifs ou suggestions et les ajouter à la base de connaissance.
- Que fait storage ?
  - Il permet de conserver les motifs appris.

17.03.26 (Soir : 21h → 01h)

> Fichier pdf convenable trouvé. Dans l’idée, j’hésite à me dire si je vais bien nettoyer la bonne base de donnée si j’extrait un pdf pour l’apprentissage python par exemple. Le fichier provient de Paris-Descartes, pour les M1 en informatique.

> Erreur rencontrée lors du test unitaire :

knowledge/nlp_extractor.py:6: in nlp = spacy.load("fr_core_news_sm")

Il manque en fait le téléchargement d’un modèle pour la langue française. Je suppose que je vais rencontrer plus tard d’autres problèmes de modèles. J’utilise donc en bash : python -m spacy download fr_core_news_sm ; et je rajoute derrière aussi le download en_core_web_sm.

> Deuxième erreur rencontrée : il faut installer le package unstructured[pdf] en plus de unstructured. Testé avec conda-forge, il est nécessaire de passer par pip install. Ce qui m’ennuie énormément vu que j’ai horreur de mélanger les gestionnaires de packages (et en plus ça prends 3 plombs).

> Troisième erreur : problème avec le regex, la complexité des algorithmes n’est pas bien capturée. Modification du code pour rendre le regex plus robuste > > Test OK !

> #### Phase de l’intégration de l’apprentissage ###

> Amélioration du storage, ajout du pattern dans knowledge_builder, et ajout dans le CLI de l’utilisation de l’apprentissage.

> Test avec un pdf : problème rencontré avec le npl_extractor. Il manque une fonction pour utiliser les extractions de blocs.

> Modification de npl_extractor pour utiliser les blocs, et ajout de son importation dans knowledge_builder.

> Je relance la ligne de commande avec le parser :

(AIGateWay) [Archene@lazychene TPIA_Gateway]$ python -m cli.main --learn-pdf tests/test_pdf.pdf Aucun fichier de motifs trouvé, démarrage avec une base vide. /home/Archene/.conda/envs/AIGateWay/lib/python3.12/site-packages/torch/cuda/**init**.py:1007: UserWarning: Can't initialize NVML raw_cnt = _raw_device_count_nvml() Traceback (most recent call last): File "", line 198, in _run_module_as_main File "", line 88, in _run_code File "/home/Archene/Dev/TPIA_Gateway/cli/main.py", line 70, in main() File "/home/Archene/Dev/TPIA_Gateway/cli/main.py", line 26, in main new = add_patterns_from_pdf(args.learn_pdf) ^^^

tests/test_pdf.pdf /home/Archene/.conda/envs/AIGateWay/lib/python3.12/site-packages/torch/cuda/**init**.py:1007: UserWarning: Can't initialize NVML raw_cnt = _raw_device_count_nvml() Aucun fichier de motifs trouvé, démarrage avec une base vide. Extraction du PDF : tests/test_pdf.pdf Warning: No languages specified, defaulting to English. Nombre de blocs extraits : 397 Blocs enrichis : 179 Ajout du motif : algorithme_inconnu Motif déjà existant : algorithme_inconnu Ajout du motif : B.P.Flannery Ajout du motif : dontd´ependleprobleme Ajout du motif : |A(e)|≤c(A)|δe| Ajout du motif : c′ Ajout du motif : siNestlatailleduprobleme(p.ex.nombred’inconnues)onpeutavoirdesalgorithmesenO(N),O(NlnN),O(N2),O(N!) Motif déjà existant : algorithme_inconnu Ajout du motif : GeorgesKOEPFLER: Motif déjà existant : algorithme_inconnu Ajout du motif : Exemple2:D´eterminationdesracinesd’ Ajout du motif : onobtientlesracinessuivantespour˜w:1,00000000010,095266145 Motif déjà existant : GeorgesKOEPFLER: Ajout du motif : Calculdelatrajectoiredelafus´eeARIANE5 Ajout du motif : DanslerapportESA Ajout du motif : BetCdesmatricesrectangulaires Ajout du motif : Algorithme1.2(Strassen)Calculduproduitdesmatricescarr´eesd’ordren=2r Ajout du motif : onchercheraxminimisant Ax−b 2.Ladernieresectionpr´esentedesm´ethodesit´erativespourlecasd’ Motif déjà existant : algorithme_inconnu Motif déjà existant : algorithme_inconnu Motif déjà existant : algorithme_inconnu Ajout du motif : IetP1telquea11soitl’´el´ementdevaleurabsoluemaximaledanslapremierecolonne Ajout du motif : . (ii)onpeutchoisirP1etP2defa¸conacequea11soitl’´el´ementdevaleurabsoluemaximaledanstoutelamatrice Ajout du motif : Siσ:{1, Motif déjà existant : algorithme_inconnu Motif déjà existant : GeorgesKOEPFLER: Ajout du motif : orpourcelailfautd´eterminer▫δA▫/▫A▫,▫δb▫/▫b▫etc(A)=▫A▫▫A−1▫.Pourobtenir▫δA▫onutiliseleth´eoremesuivantTh´eoreme2.1.3Soit˜xlasolutioncalcul´eetr=A˜x−bler´esidu Motif déjà existant : algorithme_inconnu Ajout du motif : DeplusilexisteunalgorithmedefactorisationenO(1 Ajout du motif : alorsAestsym´etriqued´e▫niepositivesietseulementsiBtABestsym´etriqued´e▫niepositive.3.UnematriceAestsym´etriqued´e▫niepositivesietseulementsiAt=Aettoutessesvaleurspropressontstrictementpositives.4.SiAestsym´etriqued´e▫niepositive Motif déjà existant : algorithme_inconnu Motif déjà existant : algorithme_inconnu Ajout du motif : Avecdesm´ethodesadapt´eesonpeutsouventam´eliorerlesperformancesdesalgorithmesdefactorisation Motif déjà existant : algorithme_inconnu Motif déjà existant : GeorgesKOEPFLER: Ajout du motif : onaHx= Ajout du motif : UΣVt Ajout du motif : deA.b)LesvaleurspropresdelamatriceAtAsontσ2i Ajout du motif : σn Motif déjà existant : UΣVt Motif déjà existant : GeorgesKOEPFLER: Ajout du motif : Onutilisealorsdesm´ethodesit´erativesoul’onarrˆetel’it´erationdesquelapr´ecisiond´esir´eeestatteinte Motif déjà existant : algorithme_inconnu Ajout du motif : resp.−U Ajout du motif : i+1aijx(k)j▫Proposition2.3.4SilamatriceAestadiagonalestrictementdominante Motif déjà existant : algorithme_inconnu Motif déjà existant : algorithme_inconnu Ajout du motif : D→Rp Ajout du motif : f=infx∈Cf(x).Ona Ajout du motif : x)= Motif déjà existant : algorithme_inconnu Ajout du motif : i.e.x∈Rn Ajout du motif : Danscecaslafonctionconˆutestlin´eairef(x)=−bipour1≤i≤p Ajout du motif : sipourtout(x Motif déjà existant : GeorgesKOEPFLER: Motif déjà existant : algorithme_inconnu Motif déjà existant : algorithme_inconnu Ajout du motif : y)∈C2,x =y Motif déjà existant : algorithme_inconnu Ajout du motif : y)∈domfetpourtoutt∈]0,1[f(tx+(1−t)y)≤max(f(x),f(y)).Onditquefeststrictementquasi Ajout du motif : CHAPITRE3.OPTIMISATIONCONTINUESANSCONTRAINTES Motif déjà existant : algorithme_inconnu Motif déjà existant : algorithme_inconnu Motif déjà existant : algorithme_inconnu Motif déjà existant :

GeorgesKOEPFLER: Ajout du motif : Exemples:1.LafonctionindicatriceICd’ Motif déjà existant : CHAPITRE3.OPTIMISATIONCONTINUESANSCONTRAINTES Motif déjà existant : GeorgesKOEPFLER: Ajout du motif : C→RconvexesurΩ,alors∇f(a)=0⇔aestunminimumglobal Motif déjà existant : algorithme_inconnu Motif déjà existant : algorithme_inconnu Ajout du motif : x=−P−1q Ajout du motif : x∈ker(P)−P+q Ajout du motif : D→Rcontinueetquasi-convexe Motif déjà existant : algorithme_inconnu Motif déjà existant : algorithme_inconnu Motif déjà existant : CHAPITRE3.OPTIMISATIONCONTINUESANSCONTRAINTES Ajout du motif : orenutilisantlad ´ecompositionLU Motif déjà existant : GeorgesKOEPFLER: Ajout du motif : Soitxtelquelescoordonn ´eesdey=Bx Ajout du motif : ∂xk(x)=n i=1sibik Motif déjà existant : CHAPITRE3.OPTIMISATIONCONTINUESANSCONTRAINTES Ajout du motif : it´e-rationsonresteO(n2),cequiestn´egligeableparrapportauco ̂utdelad ´ecompositionLU.SurlapagesuivanteonproposeunprogrammeScilab1quicomparenotreimpl ´ementationauxfonctionscondetrcondinclusesdansScilab Motif déjà existant : GeorgesKOEPFLER: Ajout du motif : E)5y Motif déjà existant : CHAPITRE3.OPTIMISATIONCONTINUESANSCONTRAINTES Motif déjà existant : algorithme_inconnu Motif déjà existant : algorithme_inconnu Ajout du motif : i.e.d∈Rn{0} tellequed/ d soitunedirectionnormalis´eedeplusgrandedescente.2)Apresavoir▫x´edilfautd ´eterminerunpasσ«convenable»:onsouhaiteavoirunedescentesigni▫cativedef(x(k))versf(x(k+1)),nepasrestertroppresdex(k),nepasmettretropdetempsatrouverσ Ajout du motif : 3.2.ALGORITHMESDEMINIMISATIONSANSCONTRAINTE Motif déjà existant : GeorgesKOEPFLER: Ajout du motif : ilexisteuneconstanteC∈R∗+telleque Ajout du motif : Sir=2onauneconvergencequadratique Ajout du motif : x▫▫−log10(C)alorsx(k+1)arfoisplusded ´ecimalesexactesquex(k).Silimk→+∞▫▫x(k+1)− Ajout du motif : comme0 Motif déjà existant : CHAPITRE3.OPTIMISATIONCONTINUESANSCONTRAINTES Ajout du motif : onsupposequefestminor´eesur{x+σd|σ≥0}.Alors Motif déjà existant : algorithme_inconnu Motif déjà existant : 3.2.ALGORITHMESDEMINIMISATIONSANSCONTRAINTE Motif déjà existant : GeorgesKOEPFLER: Ajout du motif : Algorithme3.4(Descentedegradient)choisirx(0)∈domfetposerk=0tantque▫▫∇f(x(k))▫▫>εd(k)=−∇f(x(k))d´eterminerσk>0x(k+1)=x(k)+σkd(k)k=k+1Gr ̂aceaucorollaire3.2.1lam ´ethodededescentedugradientestglobalementconvergente Motif déjà existant : algorithme_inconnu Ajout du motif : x▫▫2Q=▫1−▫▫∇f(x(k))▫▫42 Motif déjà existant : CHAPITRE3.OPTIMISATIONCONTINUESANSCONTRAINTES Ajout du motif : ouc2(Q)=λn/λ1.Ainsi Motif déjà existant : algorithme_inconnu Motif déjà existant : 3.2.ALGORITHMESDEMINIMISATIONSANSCONTRAINTE Motif déjà existant : GeorgesKOEPFLER: Ajout du motif : al’´etapek Ajout du motif : x;(ii)lam ´ethodedeNewtonestd’ Motif déjà existant : CHAPITRE3.OPTIMISATIONCONTINUESANSCONTRAINTES Motif déjà existant : algorithme_inconnu Ajout du motif : onauneconvergencetresrapide Motif déjà existant : algorithme_inconnu Ajout du motif : A−1b Ajout du motif : Posonsφ(σ)=f(x+σd),alorsφ′(σ)=0pourσ=−r(x)td Motif déjà existant : 3.2.ALGORITHMESDEMINIMISATIONSANSCONTRAINTE Motif déjà existant : GeorgesKOEPFLER: Motif déjà existant : algorithme_inconnu Motif déjà existant : algorithme_inconnu Ajout du motif : Th´eoreme3.2.4Soitx(k)▫= Ajout du motif : x Motif déjà existant : CHAPITRE3.OPTIMISATIONCONTINUESANSCONTRAINTES Ajout du motif : espacedeKrylovKk+1(A Ajout du motif : x▫2A≤▫minPk∈R[X]max1≤i≤n[1+λiPk(λi)]2▫▫x(0)− Ajout du motif : admetquervaleurspropresdistinctes0 Ajout du motif :

siλ1≈1≈λn−m<<λn−m+1,onaλn−m−λ1≈2εetλn−m+λ1≈2,d'ou x(m+1)− Ajout du motif : CxancavecCtellequeleconditionnementdelamatriceC−tAC−1soitmeilleurqueceluideA. Motif déjà existant : 3.2.ALGORITHMESDEMINIMISATIONSANSCONTRAINTE Motif déjà existant : GeorgesKOEPFLER: Ajout du motif : Casnonlin´eairePourpouvoirappliquerl' Motif déjà existant : algorithme_inconnu Motif déjà existant : algorithme_inconnu Motif déjà existant : CHAPITRE3.OPTIMISATIONCONTINUESANSCONTRAINTES Ajout du motif : f(x)=1 Ajout du motif : ∂xn(x) ∈M(m Ajout du motif : Onnoteϱj=yj−φ(x Motif déjà existant : GeorgesKOEPFLER: Motif déjà existant : algorithme_inconnu Motif déjà existant : CHAPITRE3.OPTIMISATIONCONTINUESANSCONTRAINTES Ajout du motif : Remarques:1.Laconstanteγ>0estunminimiseuruniformedesvaleurssingulieresdeJ(x):0 Ajout du motif : 2▫J(x)d+r(x)▫22,avec▫d▫≤Δ.(3.9)Supposonsquel'onsached´eterminerlasolutionde(3.9),ilfautenpluspouvoird´eter-miner,achaqueit´erationlavaleurdeΔ.Pourcelaonvaconsid´ererlerapportdelar´eductionvraieetdelar´eductiondueaumodele Motif déjà existant : algorithme_inconnu Motif déjà existant : GeorgesKOEPFLER: Ajout du motif : Δk+1=min(2Δk, Δ)sinonΔk+1=Δksiρk>ηalorsx(k+1)=x(k)+d(k)sinonx(k+1)=x(k)k=k+1Lasolutionde(3.9)estcaract´eris´eegrˆaceauLemme3.3.1LevecteurdLMestsolutiondusous-problememind1 Ajout du motif : Onaalorslimk→+∞J(x(k))tr(x(k))=0 Motif déjà existant : CHAPITRE3.OPTIMISATIONCONTINUESANSCONTRAINTES Motif déjà existant : algorithme_inconnu Ajout du motif : alors:(αu+βv)∗=αu∗+βv∗;(u∗)∗=u;(u◦v)∗=v∗ Motif déjà existant : algorithme_inconnu Ajout du motif : ◦u∗=IdE Ajout du motif : ◦ Ajout du motif : PropositionA.1.3(´Ecriturematricielle)Soit(E,<.,.>)unespaceeuclidienetB={e1, Ajout du motif : onaS=In Motif déjà existant : algorithme_inconnu Motif déjà existant : algorithme_inconnu Motif déjà existant : GeorgesKOEPFLER: Motif déjà existant : algorithme_inconnu Motif déjà existant : GeorgesKOEPFLER: Ajout du motif : Lecontextepermetdesavoirsionaa▫aireaunenormedevecteuroudematrice Ajout du motif : alors1. A 1= At ∞;2. A 2= At 2;3.SiUestorthogonale, UA 2= AU 2 = At 2;4.SiAestnormale, A 2=ρ(A)Casparticuliers Motif déjà existant : algorithme_inconnu Ajout du motif : O.2.SiIn−Aestunematricesinguli`ere Ajout du motif : O Motif déjà existant : GeorgesKOEPFLER: Ajout du motif : Onditquefestdi ´erentiableena∈Ωs' Motif déjà existant : algorithme_inconnu Ajout du motif : ∂xn(a)∂f2 Ajout du motif : ∂x1(a)∂f2 Ajout du motif : OnditaussiquelegradientestperpendiculaireauxlignesdeniveauLf(α)={x∈Rn/f(x)=α}.Si∇f(a)=O Ajout du motif : etlocalementlafonctionestplate Ajout du motif : Onditquefestcontinˆumentdi ´erentiable Ajout du motif : dgf(a)◦ Motif déjà existant : GeorgesKOEPFLER: Motif déjà existant : algorithme_inconnu Ajout du motif : C' Motif déjà existant : algorithme_inconnu Motif déjà existant : algorithme_inconnu Motif déjà existant : algorithme_inconnu 95 nouveau(x) motif(s) sauvegardé(s). 95 nouveau(x) motif(s) ajouté(s).

> Je me demande pourquoi est-ce qu'il y a des algorithmes avec pour nom : algorithme_inconnu. Je demande à l'IA qui me peaufine le NPL_extractor. Je me rends compte que le NPL n'est pas très évident.

> Modification succinte du npl_extractor et knowlegde_builder.

> Arrêt pour aujourd'hui

20.03.2026 (Matin : 11h → 13h)

> Modification de l’algorithme du knowledge_base pour y implémenter la recherche par mot-clé.

> Modification de knowledge_builder pour stocker les mots-clés

> Création d’un générateur de détecteur de mot-clé appelé detectors.py

> Modification du fichier matcher.py pour y implémenter la fonction detector.

> Modification de algorithm_advisor pour y passer le code source.

> Création d’un dossier IA pour y implémenter l’IA.

> Création du fichier classifier.py, justifié pour apprendre le croisement des données et vis à vis des données que nous allons récolter.

> Installation de nouvelles dépendances pour le projet : pytorch, transformers, scikit-learn et datasets.

> Installation reportée. 4G instable, 4 fails et 40min d’attente.

20.03.2026 (Matin : 9h → 12h)

> Installation réussie des nouvelles librairies.

> Codage de classifier.py

> Création et codage du fichier train.py pour… créer le modèle d’entraînement.

> Modification de knowledge_advisor pour y intégrer l’IA.

> Modification dans la CLI pour faire appel à l’IA.

> Je réfléchis à la manière de récupérer efficacement des données issues de scientifique/informaticien tel que Cormen.

> En demandant à l’IA, il existe déjà des jeux de données sur l’efficacité de codes et pattern en python. Je décide donc de télécharger le datasets pour y entraîner mon IA. (+ de 95000 paires de codes)

> Je code des définitions d’extraction du datasets dans un fichier nommé data_loader.py

> Le train de l’IA ne veut pas se faire. Je tente des suggestions de l’IA sur l’upgrade

> Trop de problème avec l’environnement actuel → nouvel environnement spécial pour l’entraînement de l’IA.

20.03.2026 (Soir : 00h00 → 03h00)

> Reprise du dl des librairies. Téléchargement vraiment lent.

```
The following NEW packages will be INSTALLED:

  _libgcc_mutex      pkgs/main/linux-64::_libgcc_mutex-0.1-main
  _openmp_mutex      pkgs/main/linux-64::_openmp_mutex-5.1-1_gnu
  bzip2              pkgs/main/linux-64::bzip2-1.0.8-h5eee18b_6  …
Downloading datasets-4.8.3-py3-none-any.whl (526 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 526.8/526.8 kB 17.2 kB/s  0:00:34
Downloading dill-0.4.1-py3-none-any.whl (120 kB)
Downloading fsspec-2026.2.0-py3-none-any.whl (202 kB)
Downloading multiprocess-0.70.19-py310-none-any.whl (134 kB)
Downloading accelerate-1.13.0-py3-none-any.whl (383 kB)
Downloading torch-2.10.0-3-cp310-cp310-manylinux_2_28_x86_64.whl (915.5 MB)
   ━╸━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 17.8/915.5 MB 17.4 kB/s eta 14:22:18
```

> Je réessaie avec conda.

> Désormais mon lancement d’entraînement ne trouve pas le datasets.jsonl… Le lien se trouve dans le Readme !

> En fait j’ai codé le code autour de l’extraction d’un json, mais je dois modifier mes fichiers pour pouvoir extraire du txt qui fonctionne par pair.

> Après moult péripétie dont l’inclusion d’un fichier zip qui empêche le push, j’ai galérer à effacer deux commits sans détruire ce que j’ai fait pendant les deux dernières heures.

> Les tests ont finit « concluants », je tente d’implémenter vscode.

> Yo marqué comme obsolète, pas beaucoup de documentation, je tenterais ça en dehors du temps de tp pour voir comme ça marche.

21.03.2026 (Soir : 22h00)

> Ecriture du readme et version pdf du journal.

Conclusion du projet

J’ai assez apprécier le projet, c’était intéressant de se projeter sur un chemin à suivre, voir ce qui ne marchera pas, réfléchir à des alternatives (exemple : passer d’un pdf à la recherche d’un datasets existant). J’ai malheureusement pris du retard sur mon projet dû à des circonstances personnelles (problèmes de voiture, problème de réinstallation Linux, s’occuper de son enfant, déménagement…).

J’ai pu vers la fin gagner un reboost de moral pour pouvoir au moins rendre quelque chose de « convenable ».

Par contre il a été frustrant de ne pas réussir ni à extraire les données correctement d’un pdf. C’est plutôt semé d’embuche, mais c’est une bonne expérience aussi pour voir ce qu’il faut prévoir aussi à l’avance (par exemple utiliser des librairies, devoir changer de version python, de devoir recréer même encore un environnement, conflit de version, finalement se déporter sur une nouvelle idée), et même de trouver de nouveaux intérêts ailleurs, comme pour voir par exemple comment faire une extension VsCode (que je n’ai malheureusement pas pu terminer, mais en bash ça fonctionne).